

Code Explanation

AURA Payload Parser Explanation

The provided Python module is designed to parse and transform AURA payloads. It performs a series of operations including base64 decoding, JSON extraction, progressive decoding, XML normalization, and XSLT transformation to produce the final output.

Overview

The code is structured into several functions, each responsible for a specific task in the parsing and transformation process. The main function, `parse_aura_payload`, orchestrates the entire process by calling these functions in a specific order. The module also includes a User-Defined Function (UDF) for use in Apache Spark.

Step 1: Initial Base64 Decoding

The first step involves decoding the input AURA payload from base64. This is done using the `decode_base64` function.

```
def decode_base64(encoded_str: str) -> str:
    if not encoded_str:
        return ""
    try:
        return base64.b64decode(encoded_str).decode('utf-8')
    except Exception as e:
        logger.error(f"Error decoding base64: {e}")
        return ""
```

Step 2: JSON Extraction

If the decoded string is in JSON format and contains an "interviewDetails" field, this step extracts the value of that field using the `extract_from_json` function.

```
def extract_from_json(json_str: str) -> str:
    if not json_str or not json_str.startswith("{"):
        return json_str
    try:
        json_obj = json.loads(json_str)
        if "interviewDetails" in json_obj:
            return json_obj["interviewDetails"]
        return json_str
    except Exception as e:
        logger.error(f"Error parsing JSON: {e}")
        return json_str
```

Step 3: Progressive Decoding

This step involves progressively decoding the string until it is no longer base64 encoded or until a maximum number of iterations is reached. The `progressive_decode` function is used for this purpose.

```
def progressive_decode(encoded_str: str, max_iterations: int = 100)
-> str:
```

```

    if not encoded_str:
        return ""
    result = encoded_str
    iterations = 0
    while iterations < max_iterations and not result.startswith("<")
:
        try:
            decoded = decode_base64(result)
            if not decoded or decoded == result:
                break
            result = decoded
            iterations += 1
        except Exception as e:
            logger.error(f"Error in progressive decode iteration {it
erations}: {e}")
            break
    return result

```

Step 4: XML Normalization

The decoded string is then normalized by removing the XML declaration encoding attribute and wrapping it in an XMLA container using the `normalize\_xml` function.

```

def normalize_xml(xml_str: str) -> str:
    if not xml_str or not xml_str.startswith("<"):
        return xml_str
    normalized = re.sub(r'(<?.*)encoding(.*?>)', r'12', xml_str)
    return f"<XMLA>{normalized}</XMLA>"

```

Step 5: XSLT Transformation

Based on the content of the normalized XML, an appropriate XSLT file is selected, and the XML is transformed using the `apply\_xslt\_transform` function.

```

def apply_xslt_transform(xml_str: str, xslt_path: str) -> str:
    if not xml_str or not xml_str.startswith("<"):
        return ""
    try:
        xml_doc = etree.fromstring(xml_str.encode('utf-8'))
        xslt_doc = etree.parse(xslt_path)
        transform = etree.XSLT(xslt_doc)
        result = transform(xml_doc)
        return str(result)
    except Exception as e:
        logger.error(f"Error applying XSLT transform: {e}")
        return ""

```

Step 6: Main Parsing and Transformation

The `parse\_aura\_payload` function orchestrates all the above steps. It takes the AURA input, policy number, and paths to XSLT files as input and returns a tuple containing the parsed AURA output and the policy number.

```

def parse_aura_payload(
    aura_input: str,
    policy_number: str,
    xslt_file: str,
    aura15_xslt_file: str
) -> Tuple[Optional[str], str]:
    # Initial decode
    aura_output = decode_base64(aura_input)
    # Check if JSON and extract interviewDetails if present
    if aura_output.startswith("{"):
        aura_output = extract_from_json(aura_output)
    # Progressive decoding
    aura_output = progressive_decode(aura_output)
    # Normalize XML
    normalized_xml = normalize_xml(aura_output)
    # Select XSLT based on content
    selected_xslt = aura15_xslt_file if "TXLife" in aura_output else
xslt_file
    # Apply XSLT transformation
    parsed_output = apply_xslt_transform(normalized_xml, selected_xs
lt)
    return parsed_output, policy_number

```

Step 7: Defining a Spark UDF

Finally, a UDF is defined using the `parse\_aura\_payload` function for use in Spark.

```

parse_aura_payload_udf = udf(parse_aura_payload, StringType())

```